Arrays, Slices et Maps

Objectifs

1. Observer concrètement les réallocations mémoire d'une `Slice` lors de l'utilisation de `append()` et comprendre l'intérêt de `make()`.
2. Reproduire et corriger les deux bugs les plus fréquents en Go sur les `Slices` : l'écrasement accidentel de données entre sous-slices et la rétention mémoire (*Memory Leak*).
3. Exploiter les `Maps` avec l'idiome du *"Comma Ok"*, implémenter un ensemble d'unicité (*Set*) à coût mémoire nul (`struct{}`), et trier l'affichage d'une `Map`.

Exercice 1 : Autopsie mémoire d'une `Slice` et facteur de croissance

Vous souhaitez démontrer à votre équipe l'impact des réallocations dynamiques sur le Garbage Collector lorsqu'on alimente une `Slice` sans pré-allouer sa capacité.

1. Déclarez une slice d'entiers initialement `nil` (`var donnees []int`).
2. Écrivez une boucle qui insère les entiers de `1` à `1000` un par un avec `append()`.
3. À chaque itération, détectez si la capacité `cap(donnees)` a changé par rapport à l'itération précédente. Lorsqu'un changement de capacité se produit, affichez sur une ligne :
    - Le nombre d'éléments actuels `len(donnees)`.
    - L'ancienne et la nouvelle capacité `cap(donnees)` ainsi que le facteur de multiplication.
    - L'adresse mémoire en hexadécimal du premier élément du tableau sous-jacent (`&donnees[0]` avec le verbe `%p`).
4. **Analyse et optimisation :**
    - Combien de réallocations et de copies de tableaux en mémoire se sont produites pour seulement 1000 éléments ? Que remarquez-vous sur l'adresse `&donnees[0]` à chaque fois que la capacité augmente ?
    - Créez une seconde slice `donneesOptimisees` initialisée avec `make([]int, 0, 1000)`, relancez la même boucle de détection et vérifiez qu'aucune réallocation ni changement d'adresse mémoire n'a lieu.

Exercice 2 : Le bug de corruption des sous-slices et la fuite mémoire

Dans un système financier traitant des flux d'ordres boursiers, une équipe a rencontré un bug critique en production : la modification d'une sous-liste d'ordres prioritaires corrompait silencieusement l'historique global de la journée.

Partie A : Reproduire et corriger l'écrasement par `append()`

1. Initialisez la slice représentant le carnet d'ordres de la journée :

```Go
carnetGlobal := []string{"ORD-01", "ORD-02", "ORD-03", "ORD-04", "ORD-05"}
```

2. Extrayez une sous-slice représentant les deux premiers ordres à traiter en lot :

```Go
lotPrioritaire := carnetGlobal[0:2]
```

3. Affichez `len` et `cap` de `lotPrioritaire`. Pourquoi sa capacité vaut-elle `5` alors qu'elle ne contient que `2` éléments ?
4. Ajoutez un ordre urgent dans ce lot avec `lotPrioritaire = append(lotPrioritaire, "URGENT-99")`, puis affichez `carnetGlobal`. Que constatez-vous sur l'ordre `"ORD-03"` dans `carnetGlobal` ? Expliquez mécaniquement ce qui s'est passé dans le tableau sous-jacent.
5. **Correction n°1 (Sans copie initiale) :** Modifiez uniquement la ligne d'extraction à l'étape 2 en utilisant l'opérateur de découpage à **3 indices** (`s[i:j:k]`) afin que le `append()` sur `lotPrioritaire` force l'allocation d'un nouveau tableau sans jamais altérer `carnetGlobal`. Vérifiez que `"ORD-03"` est désormais intact et comparez les adresses mémoire `&carnetGlobal[0]` et `&lotPrioritaire[0]`.

Partie B : Éliminer une fuite mémoire (*Memory Leak*)

1. Écrivez une fonction `extraireEnTete(trameBrute []byte) []byte` qui reçoit une slice de `10 Mo` (`make([]byte, 10*1024*1024)`) simulant un gros fichier réseau et qui doit retourner uniquement les **4 premiers octets** (l'en-tête du protocole).
2. Si vous écrivez simplement `return trameBrute[:4]`, quelle est la capacité (`cap`) de la slice retournée ? Pourquoi cela empêche-t-il le Garbage Collector de libérer les 10 Mo ?
3. Implémentez correctement `extraireEnTete` à l'aide de `make` et de la fonction native `copy()` pour que la slice retournée ait une capacité exacte de `4` octets, libérant ainsi totalement le reste de la trame.

Exercice 3 : Agrégation de logs, *Set* d'unicité et tri d'une `Map`

Vous devez analyser un lot brut de journaux d'accès HTTP provenant d'un répartiteur de charge (*Load Balancer*). Chaque entrée contient l'adresse IP cliente et le code de statut HTTP renvoyé :

```go
logsBruts := []string{
	"192.168.1.10:200", "10.0.0.5:500", "192.168.1.10:200",
	"172.16.0.3:404", "10.0.0.5:500", "192.168.1.10:403",
	"10.0.0.5:200", "172.16.0.3:200", "192.168.1.25:200",
	"10.0.0.5:500",
}
```

(Indice : vous pouvez découper chaque chaîne autour du caractère `":"` avec `strings.Split(entree, ":")` ou `strings.Cut(entree, ":")` du package `"strings"`).

1. **Compteur de requêtes par IP (`map[string]int`) :**
   - Parcourez `logsBruts` et comptez le nombre total de requêtes émises par chaque adresse IP dans une `map`.
2. **Détection des IP suspectes avec un *Set* zéro-octet (`map[string]struct{}`) :**
   - Toute adresse IP ayant déclenché au moins une erreur `500` ou `403` doit être enregistrée dans un ensemble d'IP sous surveillance nommé `ipSuspectes` de type `map[string]struct{}`.
   - Lorsqu'une erreur `500` ou `403` est rencontrée, insérez l'IP dans ce *Set* (utilisez `struct{}{}` comme valeur).
3. **Interrogation avec l'idiome du *"Comma Ok"* :**
   - Créez une liste d'IP à auditer : `audit := []string{"192.168.1.10", "8.8.8.8", "10.0.0.5"}`.
   - Pour chacune de ces IP, vérifiez d'abord si elle a généré du trafic dans votre première `map` (grâce à `nb, existe := ...`), puis vérifiez si elle figure dans le *Set* `ipSuspectes`.
4. **Affichage déterministe et trié :**
   - Si vous affichez directement votre `map` de compteurs avec `for ip, total := range ...` en relançant plusieurs fois le programme, vous constaterez que l'ordre d'affichage change de manière aléatoire.
   - Extrayez toutes les clés de la `map` dans une `slice` de chaînes `[]string`, triez cette slice par ordre alphabétique (explorez le package moderne `"slices"` avec `slices.Sort()` ou `"sort"`), puis affichez le rapport final de trafic dans un ordre strictement déterministe.